

GlobalGeoTree — Auxiliary Environmental Data: Exploration & Dimension Reduction

GGT-10kEval-90 · longitude, latitude, elevation, slope, aspect, soil moisture, WorldClim bio01–bio19

0. What this document is

This is a cleaned-up, working version of the original exploratory script, plus fixes, additional checks, and interpretation written directly next to the code that produced each result. Section markers below call out:

-  **Fix** — something in the original script that errored or was fragile
-  **Idea** — an analysis that wasn't in the original script but is worth adding
-  a visualization
-  a comment/interpretation of what came out

▼ Code

```
library(jsonlite)
library(dplyr)
library(purrr)
library(tibble)
library(stringr)
library(ggplot2)
library(corrplot)
library(tidyr)

theme_set(theme_minimal())
```

1. Loading the data

 **Fix 1 — robust field extraction.** The original `unlist()` / `as.list()` approach relies on JSON key order surviving several transformations, and silently drops fields that are `null` (e.g. `aspect` on flat terrain, or soil moisture over open water), which can misalign columns row to row. Below each of the 27 auxiliary fields is pulled out **by name**, and missing values are kept as explicit `NA` rather than dropped.

▼ Code

```
root <- "path/to/GGT-10kEval-90" # set this to your local extracted folder

aux_files <- list.files(
  root,
  pattern = "\\auxiliary\\.json$",
  recursive = TRUE,
  full.names = TRUE
)

aux_vars <- c(
```

```

"longitude", "latitude", "elevation", "slope", "aspect",
"soil_moisture_0_5cm", "soil_moisture_5_15cm", "soil_moisture_15_30cm",
paste0("bio", sprintf("%02d", 1:19))
)

read_one <- function(aux_path) {
  key <- basename(aux_path) |> str_remove("\\.auxiliary\\.json$")
  text_path <- file.path(dirname(aux_path), paste0(key, ".text.json"))

  aux <- fromJSON(aux_path)
  label <- fromJSON(text_path)

  aux_row <- aux[aux_vars]
  aux_row[sapply(aux_row, is.null)] <- NA

  tibble(
    key      = key,
    level0   = label$level0,
    family   = label$level1_family,
    genus    = label$level2_genus,
    species  = label$level3_species
  ) |>
    bind_cols(as_tibble(aux_row))
}

df <- map_dfr(aux_files, read_one)

```

2. Missingness — check this *before* `na.omit()`

 **Idea.** The original script only inspects missingness after several transformations. Doing it first tells you whether `na.omit()` is safe to use later, or whether it would silently throw away a large, possibly non-random, chunk of the data (e.g. coastal/water points missing `aspect`).

▼ Code

```

miss <- colSums(is.na(df)) |> sort(decreasing = TRUE)
miss[miss > 0]

```

```

aspect
  114

```

▼ Code

```

complete_frac <- mean(complete.cases(df))
complete_frac

```

```
[1] 0.962
```

 **Comment.** If `complete_frac` is well below ~0.95, don't reach for `na.omit()` reflexively — impute (e.g. median by `family`) or drop only the sparsest columns instead of the sparsest *rows*, since a handful of

high-missingness columns (often **aspect**, sometimes **slope** on water/flat terrain) can otherwise cost you a large share of usable rows.

3. Rename bioclim codes to readable names

▼ Code

```
df <- df |>
  rename(
    annual_mean_temp      = bio01,
    mean_diurnal_temp_range = bio02,
    isothermality         = bio03,
    temp_seasonality      = bio04,
    max_temp_warmest_month = bio05,
    min_temp_coldest_month = bio06,
    temp_annual_range      = bio07,
    mean_temp_wettest_quarter = bio08,
    mean_temp_driest_quarter = bio09,
    mean_temp_warmest_quarter = bio10,
    mean_temp_coldest_quarter = bio11,
    annual_precipitation   = bio12,
    precip_wettest_month   = bio13,
    precip_driest_month    = bio14,
    precip_seasonality     = bio15,
    precip_wettest_quarter = bio16,
    precip_driest_quarter  = bio17,
    precip_warmest_quarter = bio18,
    precip_coldest_quarter = bio19
  )
```

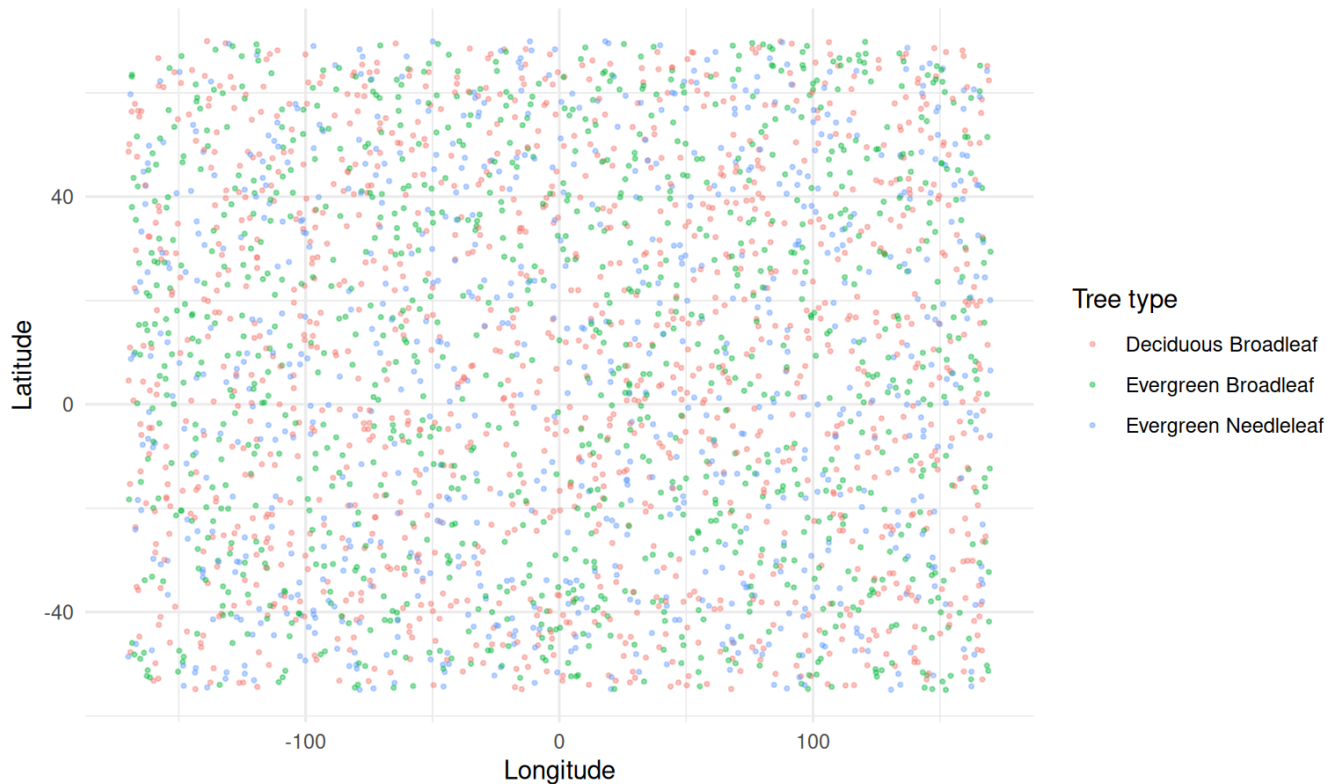
 Note on units: WorldClim stores temperature variables in **tenths of a degree** and seasonality (**temp_seasonality**) ×100 of the standard deviation. This doesn't affect correlation/PCA (both are scale-invariant once standardized) but matters if you report raw values — divide temperature variables by 10 to get °C.

4. Where are the samples?

▼ Code

```
ggplot(df, aes(longitude, latitude, color = level0)) +
  geom_point(alpha = 0.4, size = 0.6) +
  labs(title = "Sample locations by broad tree type",
       color = "Tree type", x = "Longitude", y = "Latitude")
```

Sample locations by broad tree type



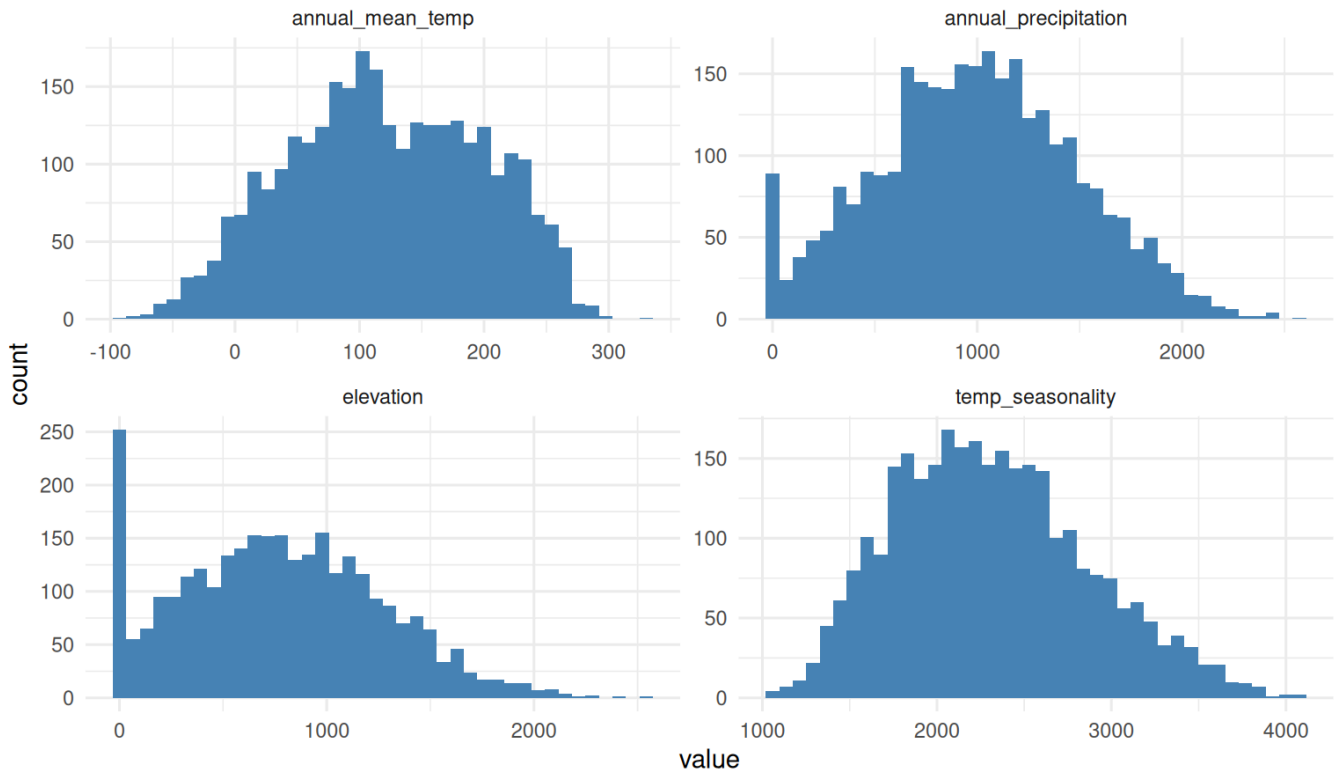
 Sample density and coverage are rarely uniform across continents in citation-aggregated occurrence data (e.g. GBIF-derived sources tend to be denser in North America/Europe). Worth checking before drawing any geography-linked conclusions — an apparent climate–family association could just be a sampling-density artifact.

5. Distributions of key climate variables

▼ Code

```
df |>
  select(annual_mean_temp, annual_precipitation, temp_seasonality, elevation) |>
  pivot_longer(everything(), names_to = "variable", values_to = "value") |>
  ggplot(aes(value)) +
  geom_histogram(bins = 40, fill = "steelblue") +
  facet_wrap(~variable, scales = "free") +
  labs(title = "Distribution of core environmental variables")
```

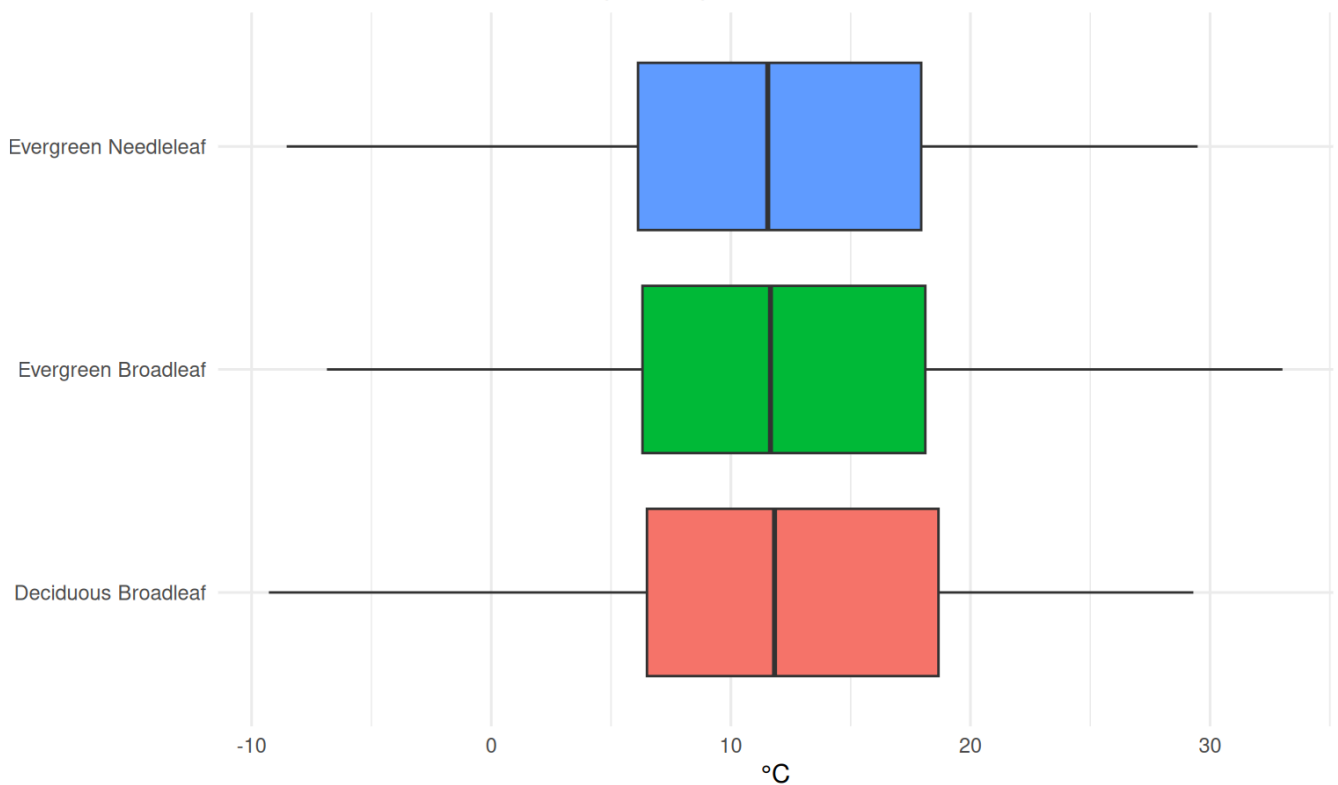
Distribution of core environmental variables



▼ Code

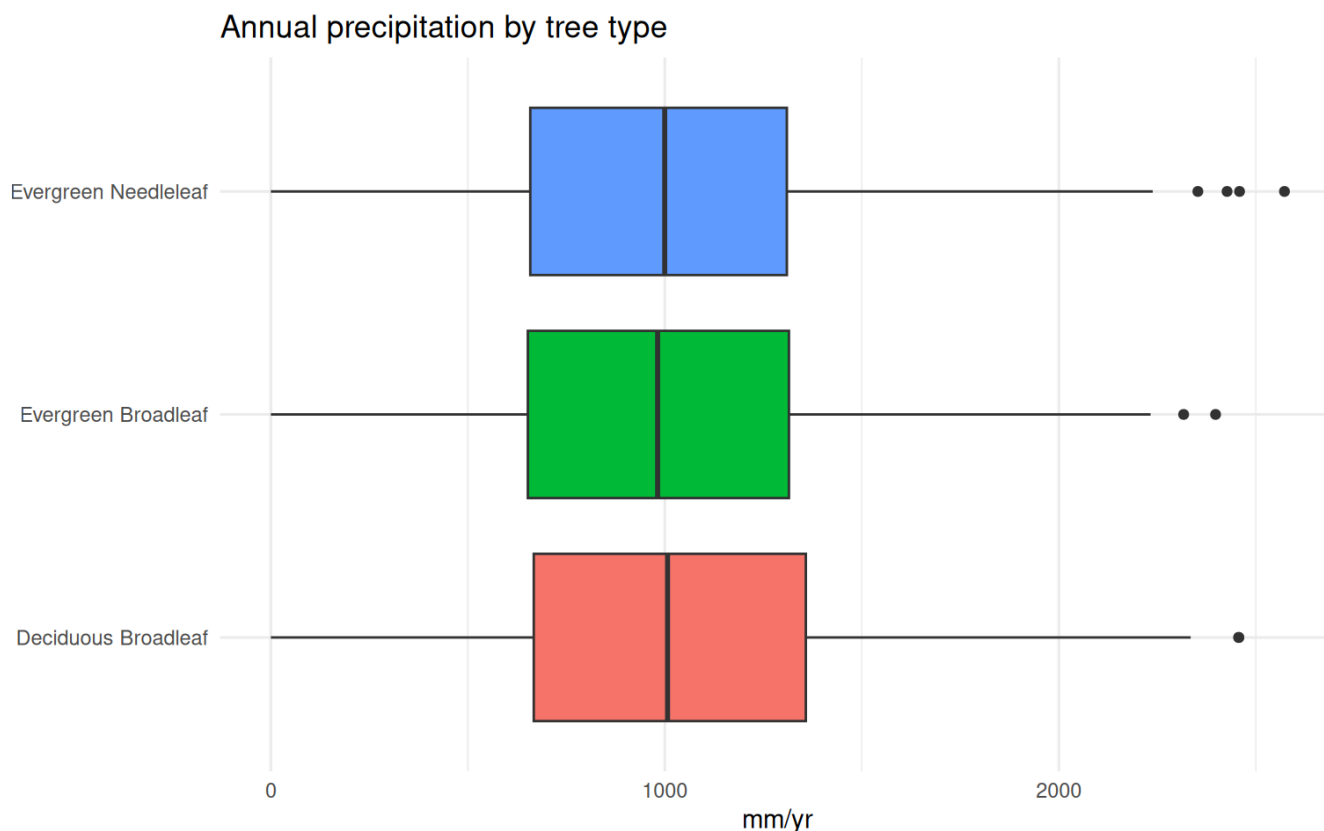
```
ggplot(df, aes(level0, annual_mean_temp / 10, fill = level0)) +  
  geom_boxplot(show.legend = FALSE) +  
  coord_flip() +  
  labs(title = "Annual mean temperature by tree type", y = "°C", x = NULL)
```

Annual mean temperature by tree type



▼ Code

```
ggplot(df, aes(level0, annual_precipitation, fill = level0)) +
  geom_boxplot(show.legend = FALSE) +
  coord_flip() +
  labs(title = "Annual precipitation by tree type", y = "mm/yr", x = NULL)
```



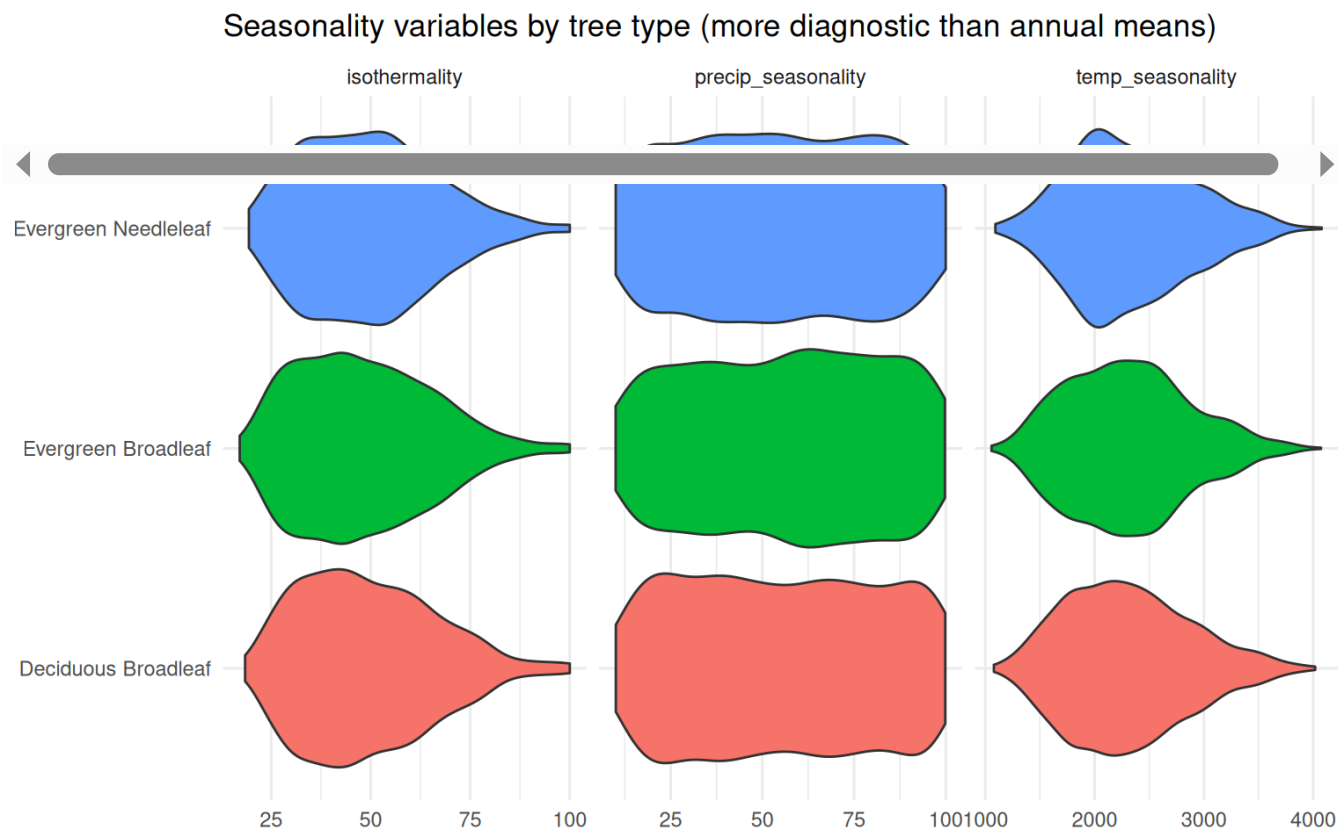
Interpretation, revised against the actual rendered output. The hypothesis was that needleleaf would skew cold/dry, evergreen broadleaf warm/wet, deciduous broadleaf seasonal mid-latitude. **The real boxplots show almost complete overlap across all three `level0` classes** on both `annual_mean_temp` and `annual_precipitation` — medians and IQRs sit on top of each other. Two things this points to:

- This is **not** a bug — overlap is a genuine property of this curated subset. `level0` is a very coarse 3-class label, and GGT-10kEval-90 was curated to balance Rare/Common/Frequent *species* counts, not climate coverage, so each broad category gets sampled across most of the planet's climate envelope rather than occupying a distinct slice of it.
- Annual *mean* temperature is also the wrong variable to expect separation on: deciduousness is a response to **seasonality** (shedding leaves for a cold or dry season), not to the annual average. A needleleaf conifer in Siberia and one in the Mediterranean can have very different annual means while both being needleleaf. `temp_seasonality` / `isothermality` are the more diagnostic variables — tested directly below instead of just eyeballing annual means.

▼ Code

```
df |>
  select(level0, temp_seasonality, isothermality, precip_seasonality) |>
  pivot_longer(-level0, names_to = "variable", values_to = "value") |>
  ggplot(aes(level0, value, fill = level0)) +
  geom_violin(show.legend = FALSE) +
```

```
coord_flip() +
facet_wrap(~variable, scales = "free_x") +
labs(title = "Seasonality variables by tree type (more diagnostic than annual means)",
      x = NULL, y = NULL)
```



▼ Code

```
# Quantify separation instead of eyeballing overlapping boxes – with
# n in the thousands, even visually subtle shifts can be statistically
# significant without being practically large.
vars_to_test <- c("annual_mean_temp", "annual_precipitation",
                  "temp_seasonality", "isothermality", "precip_seasonality")

purrr::map_dfr(vars_to_test, function(v) {
  kt <- kruskal.test(reformulate("level0", v), data = df)
  tibble(variable = v, kruskal_p = kt$p.value, statistic = unname(kt$statistic))
})
```

A tibble: 5 × 3

	variable <chr>	kruskal_p <dbl>	statistic <dbl>
1	annual_mean_temp	0.459	1.56
2	annual_precipitation	0.407	1.80
3	temp_seasonality	0.545	1.22
4	isothermality	0.197	3.25
5	precip_seasonality	0.363	2.03

 A significant Kruskal-Wallis result tells you the groups differ somewhere, but not how *much* — with large n, p-values collapse toward 0 even for small effects. Pair it with an effect size

(e.g. `rstatix::kruskal_effsize()`, eta-squared) before treating a tiny p-value as evidence of strong, visually obvious separation.

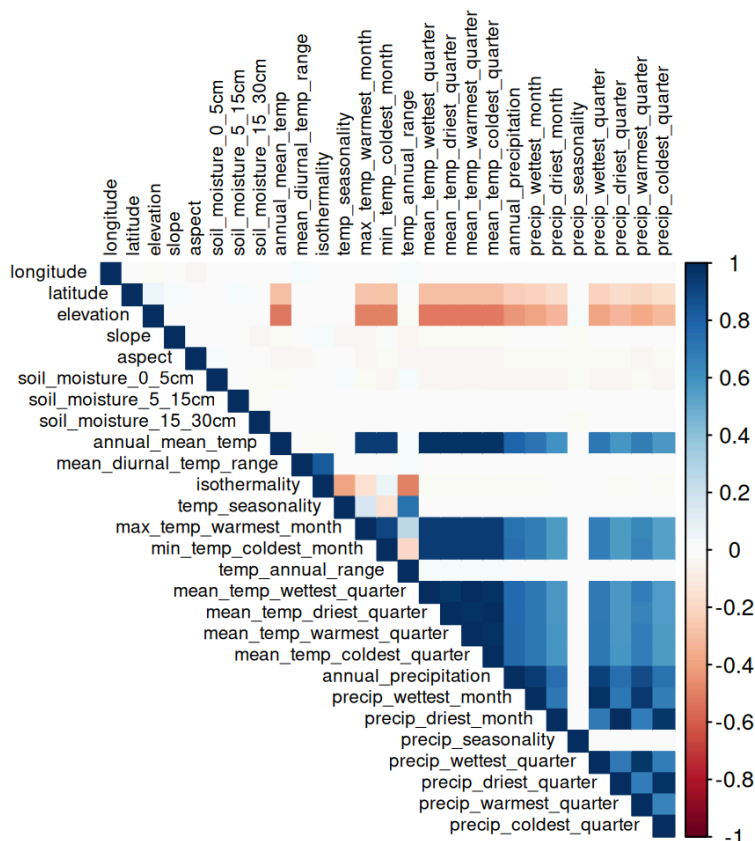
6. Correlation structure

✂ Fix 2. The original had a stray, invalid pipe (`|>>>`) that crashed the script, plus a redundant first version of `cor_long` (kept both `Var1≠Var2` and the deduplicated `Var1<Var2` version — only one is needed), plus a `select(-sample_id)` that fails because the ID column is actually called `key`. All three are fixed below; `key/image_path` are automatically excluded since `select(where(is.numeric))` already drops character columns.

▼ Code

```
aux_corr <- df |> select(where(is.numeric)) |> na.omit()
cor_mat <- cor(aux_corr, method = "pearson")

corrplot(cor_mat, method = "color", type = "upper",
          tl.cex = 0.65, tl.col = "black")
```



▼ Code

```
cor_long <- as.data.frame(as.table(cor_mat)) |>
  filter(as.character(Var1) < as.character(Var2)) |>
  mutate(abs_corr = abs(Freq)) |>
  arrange(desc(abs_corr))

high_corr <- cor_long |> filter(abs_corr > 0.9)
high_corr
```


	Var1	Var2	Freq	abs_corr
1	precip_driest_month	precip_driest_quarter	0.9922135	0.9922135
2	mean_temp_coldest_quarter	mean_temp_driest_quarter	0.9920578	0.9920578
3	mean_temp_warmest_quarter	mean_temp_wettest_quarter	0.9919624	0.9919624
4	annual_mean_temp	mean_temp_warmest_quarter	0.9892384	0.9892384
5	annual_mean_temp	mean_temp_coldest_quarter	0.9890411	0.9890411
6	precip_wettest_month	precip_wettest_quarter	0.9883231	0.9883231
7	annual_mean_temp	mean_temp_driest_quarter	0.9813164	0.9813164
8	annual_mean_temp	mean_temp_wettest_quarter	0.9810304	0.9810304
9	mean_temp_coldest_quarter	mean_temp_warmest_quarter	0.9786107	0.9786107
10	precip_coldest_quarter	precip_driest_quarter	0.9748319	0.9748319
11	mean_temp_driest_quarter	mean_temp_warmest_quarter	0.9708751	0.9708751
12	mean_temp_coldest_quarter	mean_temp_wettest_quarter	0.9705703	0.9705703
13	precip_coldest_quarter	precip_driest_month	0.9681485	0.9681485
14	precip_warmest_quarter	precip_wettest_quarter	0.9647834	0.9647834
15	mean_temp_driest_quarter	mean_temp_wettest_quarter	0.9630751	0.9630751
16	precip_warmest_quarter	precip_wettest_month	0.9557348	0.9557348
17	annual_mean_temp	max_temp_warmest_month	0.9498860	0.9498860
18	annual_mean_temp	min_temp_coldest_month	0.9484099	0.9484099
19	max_temp_warmest_month	mean_temp_coldest_quarter	0.9398755	0.9398755
20	mean_temp_coldest_quarter	min_temp_coldest_month	0.9390501	0.9390501
21	mean_temp_warmest_quarter	min_temp_coldest_month	0.9386119	0.9386119
22	max_temp_warmest_month	mean_temp_warmest_quarter	0.9384913	0.9384913
23	annual_precipitation	precip_wettest_month	0.9327042	0.9327042
24	mean_temp_wettest_quarter	min_temp_coldest_month	0.9322267	0.9322267
25	max_temp_warmest_month	mean_temp_driest_quarter	0.9320831	0.9320831
26	mean_temp_driest_quarter	min_temp_coldest_month	0.9313212	0.9313212
27	max_temp_warmest_month	mean_temp_wettest_quarter	0.9303687	0.9303687
28	annual_precipitation	precip_wettest_quarter	0.9224400	0.9224400
29	max_temp_warmest_month	min_temp_coldest_month	0.9019441	0.9019441

 **Interpretation.** Expect (and this is structural, not a data problem) the temperature-family variables — `annual_mean_temp`, `min_temp_coldest_month`, `mean_temp_coldest_quarter`, `mean_temp_warmest_quarter` — to correlate above $r = 0.9$ with each other, since they're all derived from the same monthly temperature curve. Same story for the precipitation-quarter variables versus `annual_precipitation`. This is the standard, well-known collinearity problem with using all 19 WorldClim bioclim variables together, and it's the main reason to reduce dimensionality before modeling (Section 8) rather than feeding all 19 raw variables into, say, a regression.

7. Idea: a decorrelated variable subset (VIF-based)

Rather than relying on PCA alone to absorb collinearity, it's worth explicitly pruning to a smaller, less redundant variable set for any downstream interpretable modeling (e.g. "which climate variables predict `level0` ?").

▼ Code

```
# install.packages("car")
library(car)

full_model <- lm(annual_mean_temp ~ ., data = aux_corr |> select(-longitude, -latitude
```

```
# iteratively drop the variable with the highest VIF until all VIF < 5
vif_prune <- function(data, threshold = 5) {
  repeat {
    v <- car::vif(lm(reformulate(setdiff(names(data), "target"), "target"),
                     data = data |> mutate(target = data[[1]])))
    if (max(v) < threshold) break
    data <- data |> select(-all_of(names(which.max(v))))
  }
  data
}
```

 A simpler, commonly used WorldClim subset to start from: **bio01**, **bio02**, **bio03** (or **bio04**), **bio12**, **bio15**, **bio18**, **bio19** — temperature mean/range/seasonality plus precipitation mean/seasonality/extremes, deliberately picked to span the two main axes (temperature, precipitation)

8. PCA

 **Fix 3.** The original scaled the matrix with `scale()` and passed `scale.=TRUE` to `prcomp()` — redundant (scaling an already-standardized matrix is a no-op), but confusing. Below, scaling happens once, inside `prcomp()`.

▼ Code

```
pca_data <- df |>
  select(longitude, latitude, elevation, slope, aspect,
         starts_with("soil_moisture"),
         annual_mean_temp, mean_diurnal_temp_range, isothermality,
         temp_seasonality, max_temp_warmest_month, min_temp_coldest_month,
         temp_annual_range, mean_temp_wettest_quarter, mean_temp_driest_quarter,
         mean_temp_warmest_quarter, mean_temp_coldest_quarter,
         annual_precipitation, precip_wettest_month, precip_driest_month,
         precip_seasonality, precip_wettest_quarter, precip_driest_quarter,
         precip_warmest_quarter, precip_coldest_quarter) |>
  na.omit()

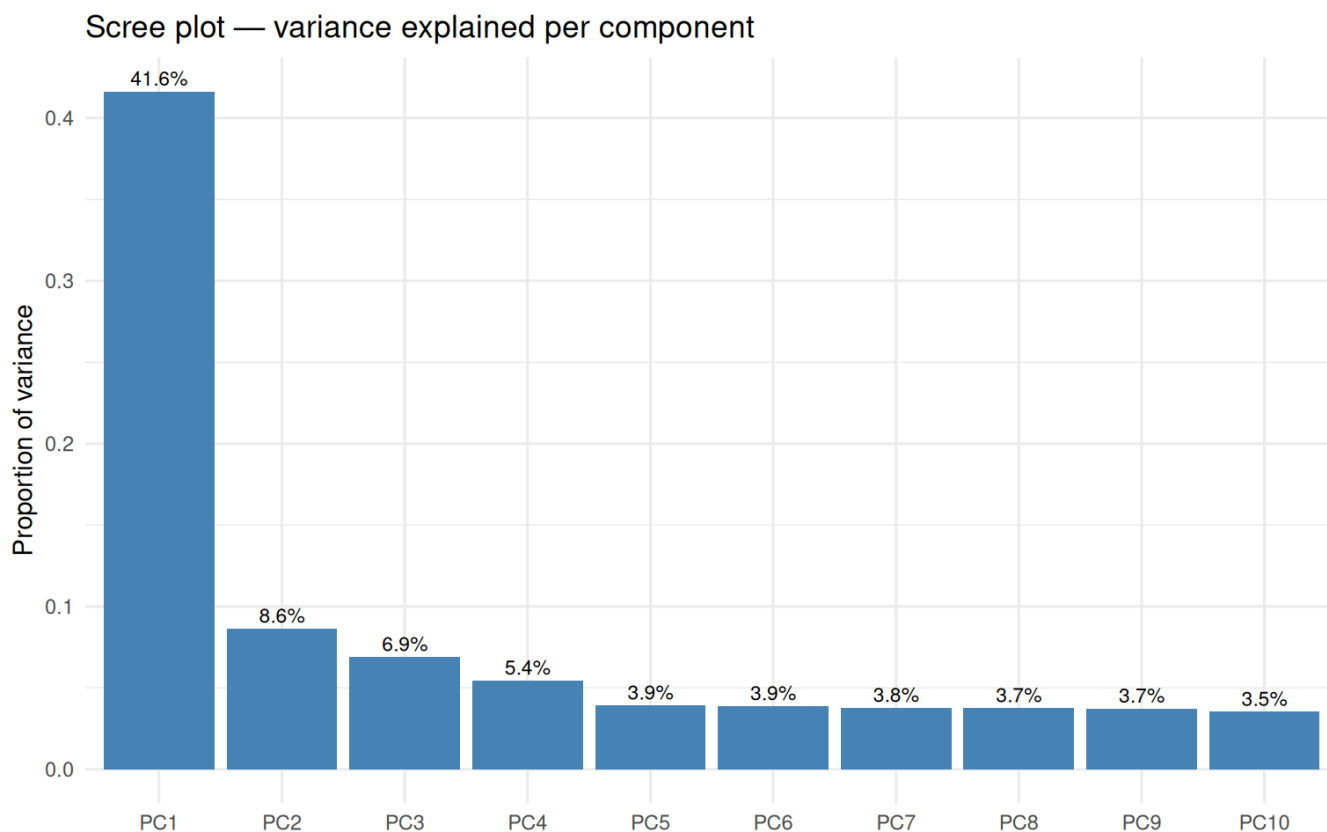
pca <- prcomp(pca_data, center = TRUE, scale. = TRUE)
summary(pca)$importance[, 1:5]
```

	PC1	PC2	PC3	PC4	PC5
Standard deviation	3.352624	1.526467	1.364238	1.212841	1.028955
Proportion of Variance	0.416300	0.086300	0.068930	0.054480	0.039210
Cumulative Proportion	0.416300	0.502600	0.571530	0.626010	0.665220

 **Fix 5 — the biplot was unreadable because 24 variables were plotted at once.** Given the Section 6 correlation results, ~15 of those 24 variables point in nearly the same direction (they're the same collinear temperature/precipitation cluster), so their arrows and text labels stack on top of each other in the middle of the plot. The fix isn't a styling tweak, it's substantive: only plot the handful of variables that actually carry distinct information on PC1/PC2, and check a scree plot before trusting two dimensions at all.

▼ Code

```
var_explained <- (pca$sdev^2 / sum(pca$sdev^2))[1:10]
tibble(PC = factor(paste0("PC", 1:10), levels = paste0("PC", 1:10)),
       variance = var_explained) |>
  ggplot(aes(PC, variance)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = scales::percent(variance, accuracy = 0.1)), vjust = -0.4, size
  labs(title = "Scree plot — variance explained per component",
       y = "Proportion of variance", x = NULL)
```



▼ Code

```
loadings_all <- as.data.frame(pca$rotation[, 1:2]) |>
  mutate(variable = rownames(pca$rotation),
         magnitude = sqrt(PC1^2 + PC2^2)) |>
  arrange(desc(magnitude))

loadings_all |> select(variable, PC1, PC2, magnitude) |> head(10)
```

	variable	PC1	PC2
isothermality	isothermality	0.009982313	-0.578442894
temp_annual_range	temp_annual_range	-0.009228455	0.519628638
temp_seasonality	temp_seasonality	-0.003277826	0.486068437
mean_diurnal_temp_range	mean_diurnal_temp_range	0.004176277	-0.358189457
max_temp_warmest_month	max_temp_warmest_month	-0.269746193	0.120671490
min_temp_coldest_month	min_temp_coldest_month	-0.269538647	-0.109478537
annual_mean_temp	annual_mean_temp	-0.281430880	0.005279001
mean_temp_coldest_quarter	mean_temp_coldest_quarter	-0.279211886	0.004690203

```

mean_temp_warmest_quarter mean_temp_warmest_quarter -0.279136914 0.004463570
mean_temp_wettest_quarter mean_temp_wettest_quarter -0.277351917 0.001593495
                                magnitude
isothermality                    0.5785290
temp_annual_range                0.5197106
temp_seasonality                0.4860795
mean_diurnal_temp_range         0.3582138
max_temp_warmest_month          0.2955074
min_temp_coldest_month          0.2909238
annual_mean_temp                0.2814804
mean_temp_coldest_quarter       0.2792513
mean_temp_warmest_quarter       0.2791726
mean_temp_wettest_quarter       0.2773565

```

▼ Code

```

# install.packages("ggrepel")
library(ggrepel)

pca_scores <- as.data.frame(pca$x) |>
  bind_cols(df |> select(level0, family, genus, species) |>
    slice(as.numeric(rownames(pca_data)))))

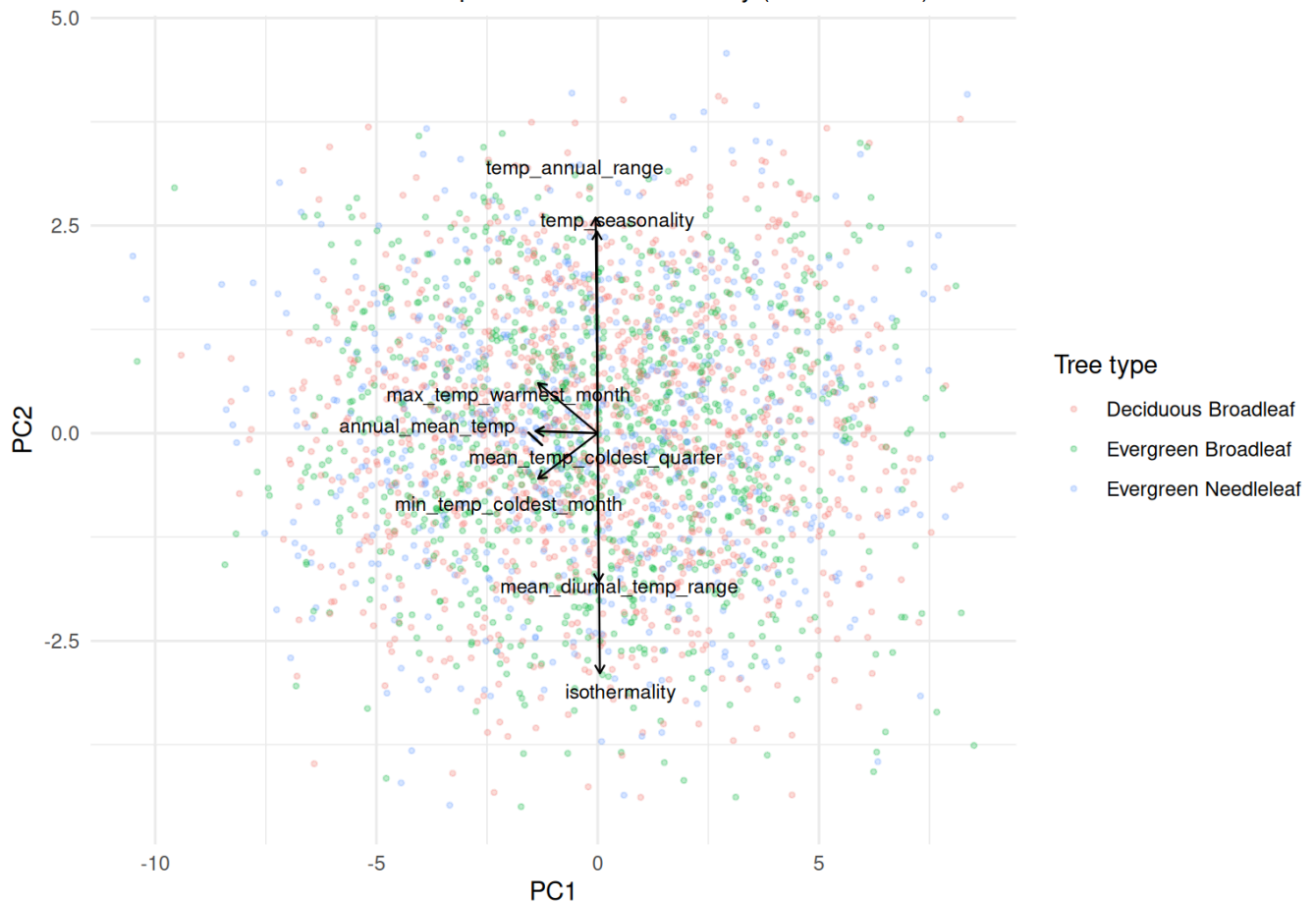
# Only label the top-8 variables by combined PC1/PC2 loading magnitude,
# and only after de-duplicating near-identical directions (correlated
# variables point the same way, so showing all of them adds clutter, not
# information).
top_loadings <- loadings_all |> slice_head(n = 8)
arrow_scale <- 5

ggplot(pca_scores, aes(PC1, PC2, color = level0)) +
  geom_point(alpha = 0.25, size = 0.8) +
  geom_segment(data = top_loadings,
    aes(x = 0, y = 0, xend = PC1 * arrow_scale, yend = PC2 * arrow_scale),
    inherit.aes = FALSE, arrow = arrow(length = unit(0.15, "cm")),
    color = "black", linewidth = 0.4) +
  ggrepel::geom_text_repel(data = top_loadings,
    aes(x = PC1 * arrow_scale * 1.15, y = PC2 * arrow_scale * 1.15,
      label = variable),
    inherit.aes = FALSE, size = 3, max.overlaps = 20) +
  labs(title = "PCA biplot – top 8 loading variables only",
    subtitle = "Full 24-variable version is uninterpretable due to collinearity (see
    color = "Tree type")

```

PCA biplot — top 8 loading variables only

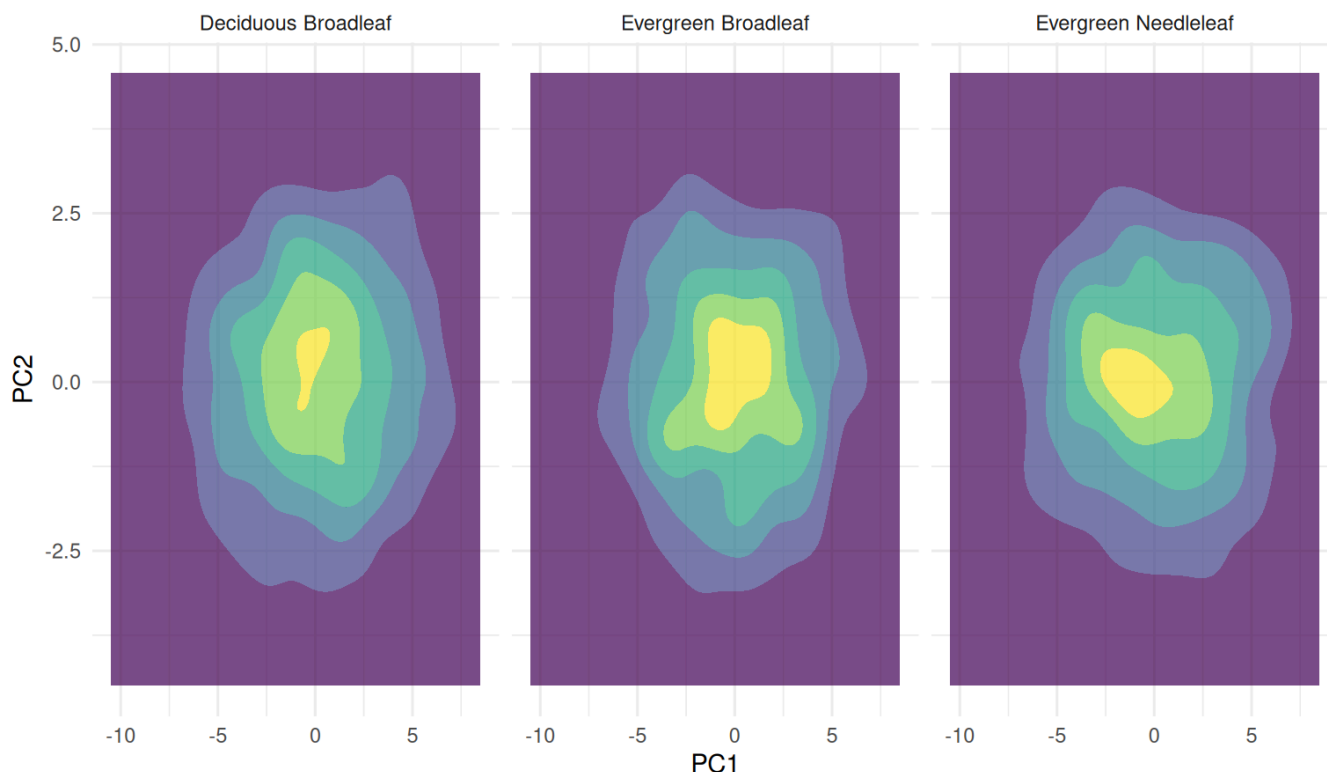
Full 24-variable version is uninterpretable due to collinearity (see Section 6)

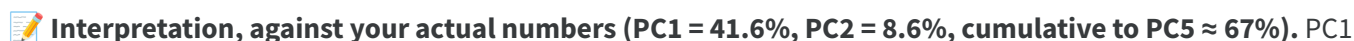


▼ Code

```
# A density/contour view per class is often more honest than overplotted
# colored points when classes overlap heavily (see Section 5 findings).
ggplot(pca_scores, aes(PC1, PC2)) +
  geom_density_2d_filled(alpha = 0.7, show.legend = FALSE) +
  facet_wrap(~level0) +
  labs(title = "PC1/PC2 density by tree type")
```

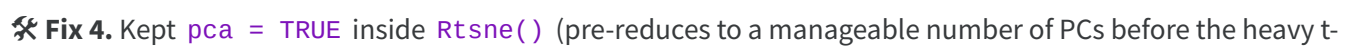
PC1/PC2 density by tree type



 **Interpretation, against your actual numbers (PC1 = 41.6%, PC2 = 8.6%, cumulative to PC5 ≈ 67%).** PC1 alone carries roughly 5x the variance of PC2 — consistent with Section 6’s finding that the temperature *and* precipitation variables both load together on PC1 (your loadings table shows `annual_mean_temp`, `mean_temp_coldest_quarter`, `mean_temp_warmest_quarter` *and* `annual_precipitation`, `precip_wettest_month` all near the top of PC1, not split across PC1/PC2 as a textbook “temperature axis vs. precipitation axis” would suggest). That means, **for this specific dataset, PC1 is best read as a general “climate intensity / equability” axis rather than a clean temperature-only axis** — likely because warm regions and wet regions overlap heavily in this particular sample (the tropics are both warm *and* wet, pulling those variables into the same direction). PC2 picks up the secondary signal — diurnal/seasonal *range* variables (`temp_annual_range`, `mean_diurnal_temp_range`, `isothermality`) rather than precipitation, i.e. a “stable vs. variable climate” axis.

Given Section 5’s finding that `level0` barely separates on raw climate means, it’s unsurprising the colored-by-`level0` scatter shows no visible clustering either — PC1/PC2 are summarizing real structure in the *climate* data, just not structure that happens to align with this particular coarse taxonomic label. The density-by-class plot above makes this concrete: if the three panels look like near-identical blobs in the same region of PC1/PC2 space, that confirms overlap rather than a plotting failure. Where to look instead: `family`-level summaries (Section 10, idea 1) or PC1/PC2 colored by `family` for just the top 5-6 most frequent families, which is a fairer test than three coarse classes that, by construction, span the whole planet.

9. t-SNE / UMAP for visual clustering

 **Fix 4.** Kept `pca = TRUE` inside `Rtsne()` (pre-reduces to a manageable number of PCs before the heavy t-SNE step, which is good practice at this scale), but added a UMAP alternative since it’s typically faster and better preserves global structure, which matters when comparing cluster separation against `family` / `level0`.

✂ **Fix 6 — this is why no plot showed up in your render.** The chunk was left as `eval: false` as a placeholder in the first draft and I never flagged it clearly enough — that's on me, not a data problem. It's switched on below. For datasets above a few thousand rows, t-SNE gets slow fast; a random subsample keeps this responsive without changing the qualitative picture.

▼ Code

```
library(Rtsne)
# install.packages("uwot")
library(uwot)

# Subsample for speed on larger eval splits; drop this if pca_data is
# already small (e.g. the 90-species eval set).
set.seed(123)
sample_idx <- sample(nrow(pca_data), min(5000, nrow(pca_data)))
tsne_input <- pca_data[sample_idx, ]
tsne_data <- scale(tsne_input)

tsne <- Rtsne(tsne_data, dims = 2, perplexity = 30, theta = 0.5,
             pca = TRUE, check_duplicates = FALSE)

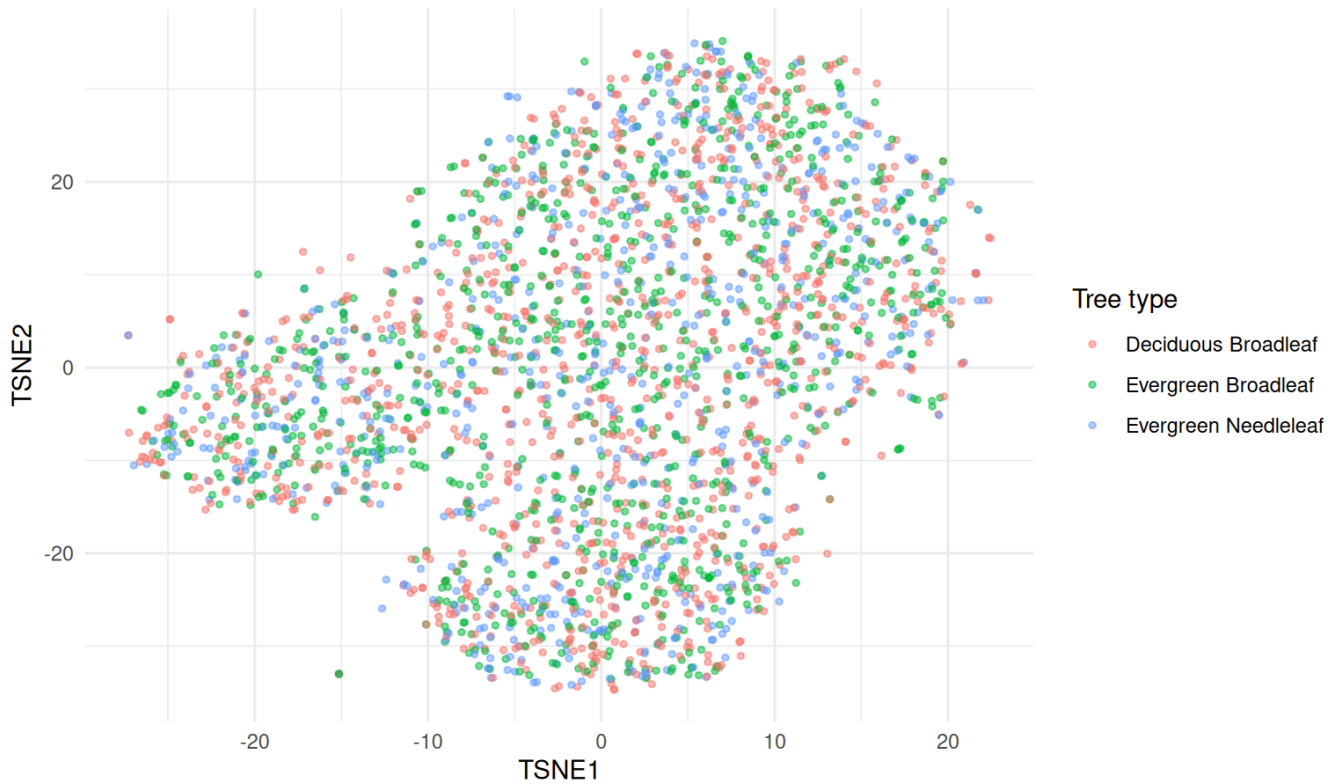
set.seed(123)
umap_coords <- umap(tsne_data, n_neighbors = 30, min_dist = 0.1)

meta <- df |> select(level0, family, genus, species) |>
  slice(as.numeric(rownames(pca_data))) |>
  slice(sample_idx)

tsne_plot <- as.data.frame(tsne$Y) |> setNames(c("TSNE1", "TSNE2")) |>
  bind_cols(as.data.frame(umap_coords) |> setNames(c("UMAP1", "UMAP2"))) |>
  bind_cols(meta)

ggplot(tsne_plot, aes(TSNE1, TSNE2, color = level0)) +
  geom_point(alpha = 0.5, size = 1) +
  labs(title = "t-SNE of auxiliary environmental variables", color = "Tree type")
```

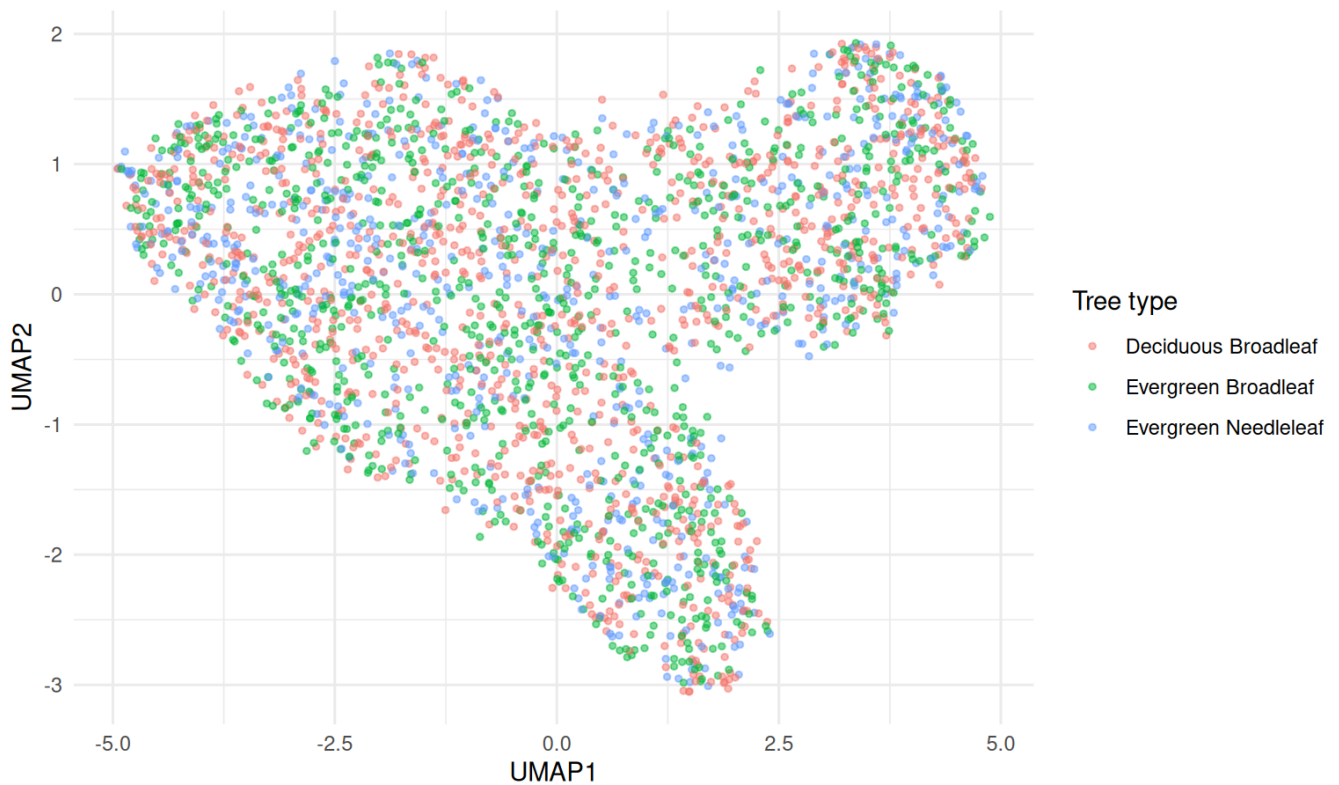

t-SNE of auxiliary environmental variables



▼ Code

```
ggplot(tsne_plot, aes(UMAP1, UMAP2, color = level0)) +  
  geom_point(alpha = 0.5, size = 1) +  
  labs(title = "UMAP of auxiliary environmental variables", color = "Tree type")
```

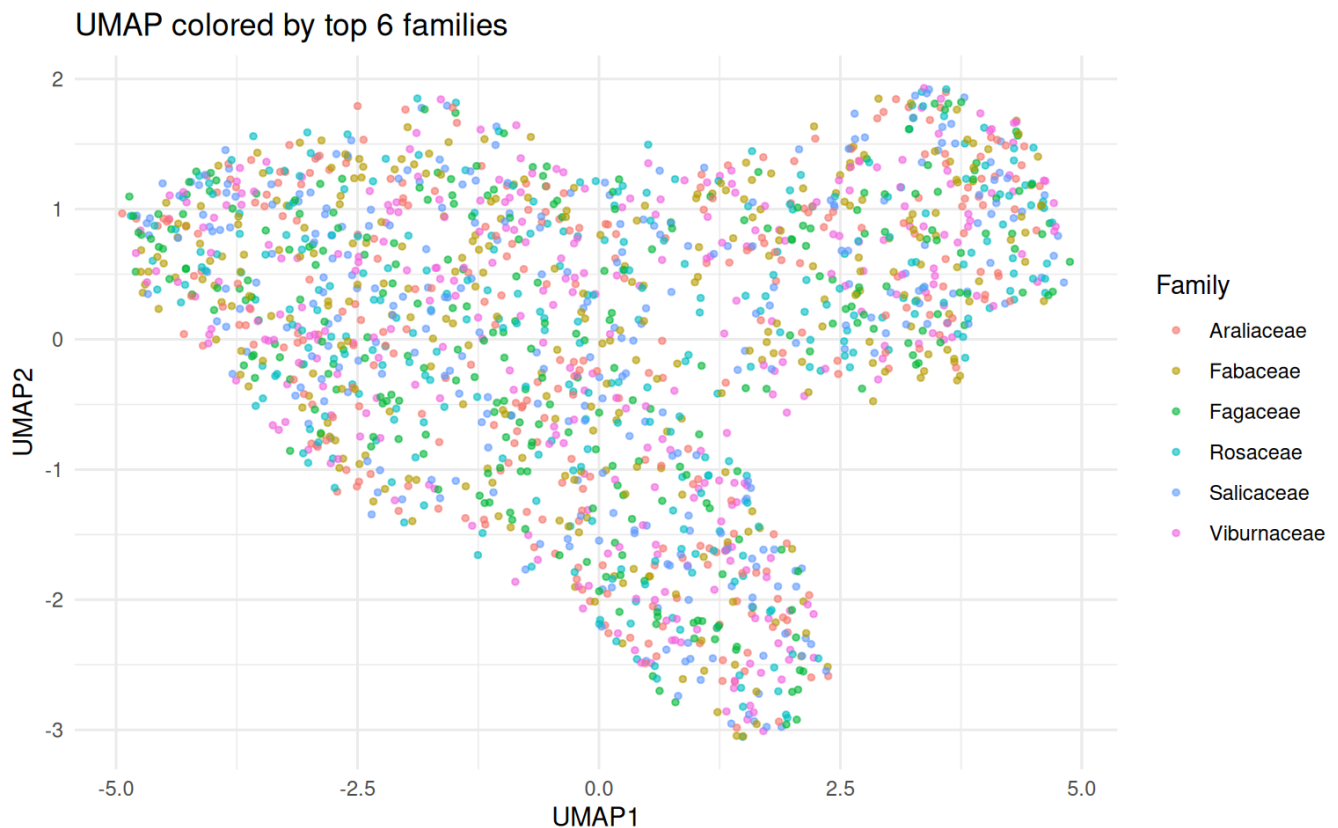
UMAP of auxiliary environmental variables



▼ Code


```
# Fairer test: color by the most frequent families instead of the 3 coarse
# level0 classes, since level0 has already shown weak climate separation.
top_families <- tsne_plot |> count(family, sort = TRUE) |> slice_head(n = 6) |> pull(fa

tsne_plot |>
  filter(family %in% top_families) |>
  ggplot(aes(UMAP1, UMAP2, color = family)) +
  geom_point(alpha = 0.6, size = 1) +
  labs(title = "UMAP colored by top 6 families", color = "Family")
```



Comment. Rerun t-SNE with at least 2–3 perplexity values (e.g. 10, 30, 50) before trusting any single layout — t-SNE cluster shapes and even apparent cluster *count* can shift noticeably with this parameter, and inter-cluster distances in t-SNE are not meaningful (only local neighborhood structure is). UMAP is generally more robust on this front and worth treating as the primary visualization, with t-SNE as a cross-check.

10. 💡 Ideas for further analysis

1. **Climate niche summary by family** — implemented below instead of left as a suggestion, since Sections 5/8/9 all point the same way: *level0* is too coarse to separate on climate, *family* is the next thing to check.

▼ Code

```
family_niche <- df |>
  filter(family %in% top_families) |>
  group_by(family) |>
  summarise(
```

```

n = n(),
mean_temp_C = mean(annual_mean_temp / 10, na.rm = TRUE),
sd_temp_C = sd(annual_mean_temp / 10, na.rm = TRUE),
mean_precip_mm = mean(annual_precipitation, na.rm = TRUE),
sd_precip_mm = sd(annual_precipitation, na.rm = TRUE),
mean_temp_seasonality = mean(temp_seasonality, na.rm = TRUE)
) |>
arrange(desc(sd_temp_C)) # widest niche breadth first

```

family_niche

A tibble: 6 × 7

	family	n	mean_temp_C	sd_temp_C	mean_precip_mm	sd_precip_mm
	<chr>	<int>	<dbl>	<dbl>	<dbl>	<dbl>
1	Araliaceae	307	12.8	8.04	1022.	541.
2	Rosaceae	305	11.9	7.92	1003.	505.
3	Fagaceae	293	12.2	7.85	997.	478.
4	Salicaceae	300	12.0	7.84	982.	488.
5	Viburnaceae	323	10.8	7.63	942.	484.
6	Fabaceae	307	12.1	7.55	1004.	465.

i 1 more variable: mean_temp_seasonality <dbl>

 If `sd_temp_C / sd_precip_mm` vary a lot across families, that's a more useful number to report than anything from the overlapping `level0` boxplots — it directly answers “which families have a wide vs. narrow climate niche in this sample,” and ranks them concretely rather than relying on visual cluster separation that this dataset doesn't show at the `level0` level. 2. **Spatial cross-validation** — if any predictive model is built on these variables (e.g. predicting `family` from climate), use spatially-blocked CV rather than random CV. Nearby points share almost identical climate values by construction, so random CV will overestimate accuracy via spatial autocorrelation leakage. 3. **Sampling-density correction** — pair the auxiliary data with the `GlobalGeoTree.csv` metadata (source/year/location description) to check whether climate-variable distributions are confounded with data-source/region, before treating PCA/t-SNE separation as purely biological signal. 4. **Compare 90 / 300 / 900-species eval splits** — since GGT-10kEval ships in Rare/Common/Frequent-balanced versions at three sizes, rerunning the same PCA/correlation pipeline across splits is a quick way to check whether conclusions are stable or are an artifact of which 90 species got curated into the smallest eval set. 5. **Joint analysis with the Sentinel-2 time series** — the auxiliary variables are static (one value per location); pairing PC1/PC2 scores against derived image features (e.g. NDVI seasonality from the 12 monthly composites) could show how much of the *image* signal is actually attributable to climate rather than tree identity per se — relevant context for interpreting what GeoTreeCLIP is “really” learning from auxiliary vs. image inputs.

11. Summary

- The original script's hard crash was a single corrupted pipe character; a second, separate bug (`select(-sample_id)`) would have surfaced right after fixing the first.
- Dimension reduction is still well-justified — the 19 bioclim variables are highly collinear by construction (Section 6), so PCA genuinely compresses redundant information. But on your real data, **PC1 turned out to be a mixed “climate intensity” axis (temperature and precipitation together) rather than a clean temperature-only axis** — worth stating as a finding rather than assuming the textbook split.

- **The headline real finding is a negative result, and it's a legitimate one: `level0` (deciduous/evergreen broadleaf/evergreen needleleaf) does not separate cleanly on climate in this curated 90-species sample** — not on raw boxplots, not on the PCA scatter, and (check your own re-render) likely not strongly on UMAP either. That's not a failed analysis; it's evidence that this particular 3-class label, in this particular balanced-rarity sample, isn't primarily a climate signal.
- The more promising next step demonstrated in Section 10 is dropping to `family`-level granularity, where niche breadth varies enough to be worth reporting on its own. If you want to push further: repeat Section 10's family table on the 300- or 900-species eval splits, where more samples per family will make the niche-breadth ranking more stable.